

ErpOne — Panduan Developer

Menambah Menu & Membangun Form (vertical slice end-to-end)

Contoh acuan: master `Brand`. Berlaku untuk fitur CRUD apa pun. Dibuat 2026-07-14

0. Vertical slice

Fitur baru menyentuh semua layer. Bangun dengan urutan ini:

1. **Domain** — entity + konfigurasi EF + migrasi.
2. **Application** — DTO, interface service, validator FluentValidation.
3. **Infrastructure** — implementasi service + registrasi DI.
4. **Web** — Index (daftar), Form (create/edit), Detail (opsional).
5. **Menu & permission** — daftarkan di `AppMenus.cs`.
6. **Test** — integration test untuk service.

Letakkan file di subfolder sesuai header menu (Master / Transactions / Finance / ...) pada tiap project. Pertahankan namespace yang ada (folder hanya untuk penataan).

Bagian A — Daftarkan menu & permission

A1 Tambahkan resource di `AppMenus.cs`

Tambahkan `AppResource` ke `ResourceGroup` yang tepat. Key menggerakkan route, permission, dan ikon secara konvensi.

```
// Web/Authorization/AppMenus.cs → di dalam grup "Master"
new("master.brands", "Brand", "bi-bookmark-star-fill", CRUD),
// key          label      ikon bootstrap      preset action
```

Preset action yang sudah ada: `CRUD` = index/create/edit/delete; `ViewOnly`, `ViewCreate`, `ReportActions` (index+export), serta preset dokumen (dengan approve/post/void). Ini otomatis membuat permission `master.brands.index|create|edit|delete`.

A2 Konvensi routing (dan override)

`NavMenuBuilder` menurunkan link otomatis:

- Href = `key.Replace('.', '/')` → `master.brands` ⇒ `/master/brands`.
- Gate visibilitas = `{key}.index`.
- Ikon = ikon deklarasi tanpa `-fill`.

Hanya jika route/action tidak beraturan, tambahkan entri ke dictionary `Overrides`:

```
// mis. route kategori berbeda; POS mengarah ke action create + link tambahan
["master.categories"]    = new(Href: "master/product-categories"),
["cashier.pos"]          = new(Href: "cashier/pos", Action: "create", Extra: [ ... ]),
```

A3 Berikan permission

`BootstrapSeeder` memberikan *semua* `AppMenus.AllPermissions` ke role admin saat startup (idempoten). Tak perlu kode.

Langkah manual setelah deploy: restart app lalu **sign out / sign in** agar klaim permission admin ter-refresh. Atur role non-admin di *Settings* → *Role*.

Bagian B — Layer Domain

B1 Entity

```
// Domain/Entities/Master/Brand.cs - namespace ErpOne.Domain.Entities
public class Brand : AuditableEntity // memberi CreatedAt/CreatedBy/...
{
    public int Id { get; private set; }
    public string Code { get; private set; } = default!;
    public string Name { get; private set; } = default!;
    public string? Description { get; private set; }

    private Brand() { } // EF
    public Brand(string code, string name, string? description)
    {
        if (string.IsNullOrEmpty(code)) throw new ArgumentException("Code wajib diisi.",
            nameof(code));
        Code = code.Trim(); Name = name.Trim(); Description = description;
    }
    public void Update(string code, string name, string? description) { /* ubah + validasi */ }
}
```

Model domain kaya: setter private, method perilaku, guard clause. Enum (status) diletakkan di file sendiri pada folder yang sama.

B2 Konfigurasi EF + migrasi

```
// Infrastructure/Persistence/AppDbContext.cs
public DbSet<Brand> Brands => Set<Brand>();
// di OnModelCreating:
modelBuilder.Entity<Brand>(e => {
    e.ToTable("M_Brands"); // prefiks M_ master / T_ transaksi
    e.HasKey(x => x.Id);
    e.Property(x => x.Code).HasMaxLength(20).IsRequired();
    e.HasIndex(x => x.Code).IsUnique();
    e.Property(x => x.Name).HasMaxLength(100).IsRequired();
});
```

```
dotnet ef migrations add AddBrand --project src/ErpOne.Infrastructure --startup-project
src/ErpOne.Web
dotnet ef database update --project src/ErpOne.Infrastructure --startup-project
src/ErpOne.Web
```

Bagian C — Layer Application

C1 DTO

```
// Application/Master/Brands/BrandDtos.cs - namespace ErpOne.Application.Brands
public record BrandDto(int Id, string Code, string Name, string? Description, DateTime CreatedAt,
string? CreatedBy);
public record CreateBrandRequest(string Code, string Name, string? Description);
public record UpdateBrandRequest(string Code, string Name, string? Description);
```

C2 Interface service

```
public interface IBrandService
{
    Task<IReadOnlyList<BrandDto>> GetAllAsync(CancellationTokens ct = default);
    Task<PagedResult<BrandDto>> GetPagedAsync(int page, int pageSize, string? search = null,
CancellationTokens ct = default);
    Task<BrandDto?> GetByIdAsync(int id, CancellationTokens ct = default);
    Task<BrandDto> CreateAsync(CreateBrandRequest request, CancellationTokens ct = default);
    Task<bool> UpdateAsync(int id, UpdateBrandRequest request, CancellationTokens ct = default);
    Task<bool> DeleteAsync(int id, CancellationTokens ct = default);
}
```

C3 Validator (FluentValidation)

```
public class CreateBrandValidator : AbstractValidator<CreateBrandRequest>
{
    public CreateBrandValidator()
    {
        RuleFor(x => x.Code).NotEmpty().MaximumLength(20)
            .Matches("^[A-Za-z0-9-]+$").WithMessage("Code hanya boleh huruf, angka, dan strip.");
        RuleFor(x => x.Name).NotEmpty().MaximumLength(100);
        RuleFor(x => x.Description).MaximumLength(300);
    }
}

// + UpdateBrandValidator. Otomatis diregistrasi via
AddValidatorsFromAssemblyContaining<CreateProductValidator>().
```

Bagian D — Layer Infrastructure

D1 Implementasi service

```
// Infrastructure/Services/Master/BrandService.cs - namespace ErpOne.Infrastructure.Services
public class BrandService(
    AppDbContext db,
    IValidator<CreateBrandRequest> createValidator,
    IValidator<UpdateBrandRequest> updateValidator) : IBrandService // DI primary-ctor
{
    public async Task<PagedResult<BrandDto>> GetPagedAsync(int page, int pageSize, string? search = null, CancellationToken ct = default)
    {
        page = Math.Max(1, page); pageSize = Math.Clamp(pageSize, 1, 200);
        var q = db.Brands.AsNoTracking();
        if (!string.IsNullOrEmpty(search))
            q = q.Where(x => x.Name.Contains(search) || x.Code.Contains(search));
        var total = await q.CountAsync(ct);
        var items = await q.OrderBy(x => x.Name).Skip((page-1)*pageSize).Take(pageSize)
            .Select(x => ToDto(x)).ToListAsync(ct);
        return new PagedResult<BrandDto>(items, total, page, pageSize);
    }

    public async Task<BrandDto> CreateAsync(CreateBrandRequest req, CancellationToken ct = default)
    {
        await createValidator.ValidateAndThrowAsync(req, ct); // melempar ValidationException
        var e = new Brand(req.Code, req.Name, req.Description);
        db.Brands.Add(e); await db.SaveChangesAsync(ct);
        return ToDto(e);
    }

    // GetAll/GetById/Update/Delete serupa; selalu AsNoTracking untuk baca.
    private static BrandDto ToDto(Brand x) => new(x.Id, x.Code, x.Name, x.Description, x.CreatedAt, x.CreatedBy);
}
```

D2 Registrasi di DI

```
// Infrastructure/DependencyInjection.cs
services.AddScoped<IBrandService, BrandService>();
```

Bagian E — Web: Halaman Index (daftar)

Gunakan pola master-index `.pi` : crumbs + KPI (opsional) + toolbar (search / filter / chips) + card tabel + `<Pager>` . Kunci route dengan policy `.index` .

```
// Web/Components/Pages/Master/Brands/BrandIndex.razor
@page "/master/brands"
@attribute [Authorize(Policy = "master.brands.index")]
@rendermode InteractiveServer
@Inject IBrandService BrandService

<div class="pi">
  <div class="pi-head"> /* crumbs + judul + tombol New (AuthorizeView create) */ </div>
  <div class="toolbar">
    <div class="search"><i class="bi bi-search"></i>
      <input placeholder="Cari..." @bind="_search" @bind:event="oninput" @onkeyup="ReloadAsync" />
    </div>
  </div>
  @* card + <table> atas _page.Items, lalu <Pager Page= TotalPages= OnPageChanged= /> *@
</div>

@code {
  private const int PageSize = 15;
  private PagedResult<BrandDto>? _page; private string? _search; private int _currentPage = 1;
  protected override Task OnInitializedAsync() => LoadAsync();
  private async Task LoadAsync() => _page = await BrandService.GetPagedAsync(_currentPage, PageSize,
    _search);
  private async Task ReloadAsync() { _currentPage = 1; await LoadAsync(); }
  private async Task GoToPageAsync(int p) { _currentPage = p; await LoadAsync(); }
}
```

Untuk filter status pakai chips: `<div class="chips"><button class="chip @(on?"on":"","")" @onclick="...">...</button></div>` (lihat contoh nyata di halaman Stock Levels).

Bagian F — Web: Halaman Form (create/edit)

Form master memakai container `.cf` (card + grid responsif). Satu route menangani create dan edit. Cek ulang permission di `OnInitializedAsync` . Petakan error `ValidationException` ke pesan per-field.

```
// Web/Components/Pages/Master/Brands/BrandForm.razor
@page "/master/brands/new"
@page "/master/brands/{Id:int}/edit"
@attribute [Authorize]
@rendermode InteractiveServer
@using FluentValidation
@inject IBrandService BrandService
@inject IAuthorizationService Auth
@inject NavigationManager Nav

<div class="cf">
    <div class="cf-top"><div class="crumbs"> ... </div></div>
    <div class="cf-wrap">
        <section class="card">
            <div class="card-h"><span class="hd-ic"><i class="bi bi-award"></i></span>
                <div class="hd-tx"><h2>Informasi brand</h2><p>...</p></div></div>
            <div class="card-b">
                <div class="grid">
                    <div class="f c4"><label class="fl">Code <span class="req">*</span></label>
                        <input class="ctl mono up @(_codeError!=null?"bad":"")" @bind="_code"
@bind:event="oninput" />
                        <div class="err-txt">@_codeError</div></div>
                    <div class="f c8"> ... Name ... </div>
                    <div class="f c12"> ... Description (textarea) ... </div>
                </div>
            </div>
        </section>
        <div class="pf-footer"><div class="in">
            <a class="btn btn-ghost" href="/master/brands">Batal</a>
            <button class="btn btn-primary" @onclick="SaveAsync" disabled="@_saving">Simpan brand</button>
        </div></div>
    </div>

@code {
    [Parameter] public int? Id { get; set; }
    [CascadingParameter] private Task<AuthenticationState> AuthStateTask { get; set; } = default!;
    private string _code="", _name=""; private string? _description, _error, _codeError, _nameError;
    private bool _loading=true, _saving, _notFound;

    protected override async Task OnInitializedAsync() {
        var user = (await AuthStateTask).User;
        var perm = Id is null ? AppMenus.Perm("master.brands","create") :
AppMenus.Perm("master.brands","edit");
        if (!(await Auth.AuthorizeAsync(user, perm)).Succeeded) { Nav.NavigateTo("/master/brands");
return; }
        if (Id is int id) { var b = await BrandService.GetByIdAsync(id);
            if (b is null) _notFound = true; else { _code=b.Code; _name=b.Name; _description=b.Description;
} }
        _loading = false;
    }

    private async Task SaveAsync() {
        _error=_codeError=_nameError=null; _saving=true;
        try {
```

```

        if (Id is int id) await BrandService.UpdateAsync(id, new
UpdateBrandRequest(_code,_name,_description));
        else
            await BrandService.CreateAsync(new
CreateBrandRequest(_code,_name,_description));
        Nav.NavigateTo("/master/brands");
    }
    catch (ValidationException ex) {
        // petakan error per-field
        foreach (var e in ex.Errors) {
            if (e.PropertyName == nameof(CreateBrandRequest.Code)) _codeError = e.ErrorMessage;
            else if (e.PropertyName == nameof(CreateBrandRequest.Name)) _nameError = e.ErrorMessage;
            else _error = e.ErrorMessage;
        }
    }
    finally { _saving = false; }
}
}

```

Kelas grid & kontrol form (.cf)

Kelas	Fungsi
<code>.grid + .f .cN</code>	Grid responsif 12-kolom; satu field membentang <code>c4 / c8 / c12</code> dst.
<code>.fl / .req</code>	Label field / tanda wajib (*).
<code>.ctl (+ .mono .up .bad)</code>	Input/select/textarea; <code>.up</code> huruf besar, <code>.bad</code> = border error.
<code>.err-txt / .cnt</code>	Pesan error inline / penghitungan karakter.
<code>.pf-footer .in</code>	Bar aksi menempel (Batal / Simpan).

Form transaksi (dengan tabel baris item) memakai `<div class="pf pf--form">` dan tabel `.items`, bukan `.cf`. Halaman detail memakai `<div class="pf pf--detail">` dengan info-grid + `.items` + `.badge` status. Lihat `PoForm / PoDetail`.

Bagian G — Test

Integration test service terhadap SQLite. Fixture membangun skema; seed lewat service atau DbContext.

```
public class BrandServiceTests : IClassFixture<CustomWebApplicationFactory>
{
    private readonly CustomWebApplicationFactory _f;
    public BrandServiceTests(CustomWebApplicationFactory f) { _f = f; _f.InitializeDatabase(); }

    [Fact] public async Task Create_lalu_get_roundtrip() {
        using var scope = _f.Services.CreateScope();
        var svc = scope.ServiceProvider.GetRequiredService<IBrandService>();
        var dto = await svc.CreateAsync(new CreateBrandRequest("NKE", "Nike", null));
        Assert.Equal("Nike", (await svc.GetByIdAsync(dto.Id))!.Name);
    }
}
```

SQLite = satu koneksi in-memory yang dibagi dalam satu kelas test (terisolasi antar-kelas). Hindari translasi EF khusus SQL Server. Hentikan app yang berjalan sebelum `dotnet test` (test mereferensi `ErpOne.Web`).

Bagian H — Definition of done (checklist)

- ☐ Entity + konfigurasi EF + migrasi diterapkan.
- ☐ DTO, interface service, validator, implementasi service, registrasi DI.
- ☐ Resource di `AppMenus.cs` ditambahkan (+ `Overrides` jika route/action tak beraturan).
- ☐ Halaman Index (`.pi`), Form (`.cf` / `.pf-form`), Detail (`.pf-detail`) bila perlu; semua dengan `[Authorize(Policy)]` + `AuthorizeView` pada aksi.
- ☐ Integration test hijau; `dotnet build ErpOne.slnx` = 0 error.
- ☐ Restart + sign out/in agar admin mendapat permission baru.
- ☐ File baru diletakkan di subfolder header menu yang benar (namespace tetap).